Pointers to Arrays

1. Address Reference:

Pointers to arrays store the memory address of the first element in the array.

2. Pointer Arithmetic:

Allows for efficient navigation through array elements using pointer arithmetic, enhancing flexibility in array manipulation.

3. Type Matching:

Require matching types between the pointer and array elements for proper addressing and interpretation of data.

4. Memory Efficiency:

Offer a memory-efficient approach, particularly useful for dynamically allocated arrays and multidimensional arrays, enhancing program performance.

Code Snippet 1 (Incorrect Type Example)

```C/C++
#include <stdio.h>
int main()
{
    int arr[4] = {1, 2, 3, 4};
    char *ptr = arr;   // ❌ Wrong type (int* expected)
    return 0;
}
```

👉 **Note:**
Pointer type must match array type → should be `int *ptr`

Code Snippet 2 (Pointer with Array Traversal)

```c
C/C++
#include <stdio.h>
int main()
{
    int i;
    int arr[4] = {1, 2, 3, 4};
    int *ptr = arr;
    for(i = 0; i < 4; i++)
        printf("%d ", *(ptr + i));  // pointer arithmetic
    return 0;
}
```

Pointer Arithmetic Explanation

- `ptr = arr` → points to first element
- `*ptr` → value at first element
- `*(ptr + 1)` → second element
- `*(ptr + 2)` → third element

Memory Representation Example

Array:

arr = [5, 8, 2]

Memory:

Address	Value
2000	5
2004	8
2008	2

Pointer Operations:

ptr = arr → 2000
*ptr → 5
*(ptr + 0) → 5
*(ptr + 1) → 8
*(ptr + 2) → 2

Code Snippet 3 (String Copy using Pointer)

```C/C++
#include <stdio.h>
#include <string.h>
int main()
{
    char dest[20];
    char *src = "world";
    strcpy(dest, src);   // Copies "world" into dest
    printf("%s", dest);
    return 0;
}
```

Pointer to Structure Variable

Basic Structure Declaration

```C/C++
struct abc {
    int x;
    int y;
};
```

Pointer to Structure Example

```C/C++
#include <stdio.h>
int main() {
    struct abc a = {0, 1};
    struct abc *ptr = &a;
    printf("%d %d", ptr->x, ptr->y);
    return 0;
}
```

👉 **Explanation:**

- `ptr` is a pointer to structure `abc`
- `&a` gives address of structure variable `a`
- `ptr->x` is shorthand for `(*ptr).x`

Accessing Structure Members via Pointers

Arrow Operator (->):

- Used to access structure members using pointers
- Shortcut for `(*ptr).member`

Key Points:

- Pointer stores address of structure
- Enables direct access to members
- Supports pointer arithmetic (arrays of structures)
- Useful in dynamic memory allocation

Example: Structure with Pointer

```c
#include <stdio.h>
#include <string.h>
struct Person {
    char name[50];
    int age;
    float height;
};
int main() {
    struct Person person1;
    person1.age = 25;
    person1.height = 175.5;
    strcpy(person1.name, "John Doe");
    struct Person *ptrPerson = &person1;
    printf("Name: %s\n", ptrPerson->name);
    printf("Age: %d\n", ptrPerson->age);
    printf("Height: %.2f\n", ptrPerson->height);
    return 0;
}
```

Example with Multiple Structures

```c
#include <stdio.h>
#include <string.h>
struct Person {
    char name[50];
    int age;
    float height;
};
int main() {
    struct Person person1, person2;
    person1.age = 25;
    person1.height = 175.5;
```

```
    strcpy(person1.name, "John Doe");
    person2.age = 26;
    person2.height = 190;
    strcpy(person2.name, "mary");
    struct Person *ptrPerson = &person2;
    printf("Name: %s\n", ptrPerson->name);
    printf("Age: %d\n", ptrPerson->age);
    printf("Height: %.2f\n", ptrPerson->height);
    return 0;
}
```

Dynamic Memory Allocation with Structures

Concepts:

- Allocate memory using `malloc()` / `calloc()`
- Use pointer to access structure
- Free memory using `free()`

Dynamic Structure Example

```C/C++
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct Student {
    char name[50];
    int age;
    float gpa;
};
int main() {
    struct Student *studentPtr;
```

```
    studentPtr = (struct Student *)malloc(sizeof(struct
Student));
    if (studentPtr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }
    strcpy(studentPtr->name, "John Doe");
    studentPtr->age = 20;
    studentPtr->gpa = 3.8;
    printf("Student Name: %s\n", studentPtr->name);
    printf("Student Age: %d\n", studentPtr->age);
    printf("Student GPA: %.2f\n", studentPtr->gpa);
    free(studentPtr);
    return 0;
}
```

Pointers to Arrays of Structures

Array of Structures:

- Group multiple structure variables in a single array.
- Used for storing related data collectively.

Pointer to Array of Structures:

- A pointer can store the base address of the array of structures.

Access via Pointer Arithmetic:

- Navigate through elements using pointer arithmetic.

Member Access with Arrow Operator:

- Use `->` to access structure members via pointer.

Example: Array of Structures with Pointer

```C/C++
#include <stdio.h>
#include <string.h>
struct Person {
    char name[50];
    int age;
    float height;
};
int main() {
    struct Person people[3];
    strcpy(people[0].name, "Alice");
    people[0].age = 28;
    people[0].height = 165.0;
    strcpy(people[1].name, "Bob");
    people[1].age = 35;
    people[1].height = 180.5;
    strcpy(people[2].name, "Charlie");
    people[2].age = 22;
    people[2].height = 170.2;
    struct Person *ptrPeople = people;
    for (int i = 0; i < 3; i++) {
        printf("Person %d:\n", i + 1);
        printf("Name: %s\n", (ptrPeople + i)->name);
        printf("Age: %d\n", (ptrPeople + i)->age);
        printf("Height: %.2f\n", (ptrPeople + i)->height);
        printf("\n");
    }
    return 0;
}
```

Pointers to Functions Returning Arrays

Concepts:

- Efficient memory usage (no copying of arrays)
- Avoid fixed-size limitations
- Modify original data directly
- Reduce time complexity

❌ Incorrect Example (Important)

```C/C++
int *getarray() {
    int arr[5];
    // input values
    return arr;   // ❌ WRONG (local array)
}
```

👉 **Problem:**

- `arr` is local → memory is destroyed after function ends
- Leads to **undefined behavior**

✅ Correct Approach (Dynamic Memory)

```C/C++
#include <stdio.h>
#include <stdlib.h>
int* getarray() {
    int *arr = (int*)malloc(5 * sizeof(int));
    for(int i = 0; i < 5; i++) {
        scanf("%d", &arr[i]);
    }
```

```
    return arr;
}
int main() {
    int *n = getarray();
    printf("Elements are:\n");
    for(int i = 0; i < 5; i++) {
        printf("%d ", n[i]);
    }
    free(n);
    return 0;
}
```

Passing Array to Function using Pointer

```C/C++
#include <stdio.h>
void print_val(int *ptr) {
    for(int i = 0; i < 5; i++)
        printf("%d ", *(ptr + i));
}
int main() {
    int arr[] = {8, 9, 7, 9, 10};
    print_val(arr);
    return 0;
}
```

Pointers to Functions Returning Structures

Concepts:

- Memory efficient (no copying of structures)

- Dynamic allocation possible
- Avoid structure duplication

Example: Returning Structure via Pointer

```C/C++
#include <stdio.h>
#include <stdlib.h>
struct Point {
    int x;
    int y;
};
struct Point* createPoint(int x, int y) {
    struct Point* point = (struct Point*)malloc(sizeof(struct
Point));
    if (point != NULL) {
        point->x = x;
        point->y = y;
    }
    return point;
}
```

Program: Pointer to Structure (Date Example)

```C/C++
#include <stdio.h>
#include <stdlib.h>
struct date {
    int d;
    int m;
    int y;
};
void disp_date(struct date *dp)
{
```

```c
    printf("%d-%d-%d", dp->d, dp->m, dp->y);
}
int main()
{
    struct date d1, *ptr;
    d1.d = 2;
    d1.m = 3;
    d1.y = 2024;
    ptr = &d1;
    disp_date(ptr);
    return 0;
}
```

Common Pitfalls & Best Practices – Arrays

1. Buffer Overflow

- **Pitfall:** Accessing beyond array size → undefined behavior
- **Mitigation:** Always check array bounds

2. Missing Null Terminator (Strings)

- **Pitfall:** Forgetting `'\0'` causes unexpected results
- **Mitigation:** Allocate enough space and properly initialize strings

3. Dangling Pointers

- **Pitfall:** Using pointer after memory is freed
- **Mitigation:** Set pointer to `NULL` after `free()`

Common Pitfalls & Best Practices – Structures

1. Incomplete Initialization

- **Pitfall:** Not initializing all members → unpredictable results

- **Mitigation:** Initialize all fields properly

2. Memory Leaks

- **Pitfall:** Forgetting `free()` for dynamic memory
- **Mitigation:** Always release memory when done

3. Nested Structure Deallocation

- **Pitfall:** Not freeing inner allocated memory
- **Mitigation:** Free memory recursively

Real-World Applications

- **Dynamic Memory Allocation**
- **Dynamic Data Structures** (Linked Lists, Trees)
- **File Handling**
- **Database Programming**
- **Signal Processing**

Conclusion

1. Dynamic memory allocation improves flexibility
2. Efficient passing of data to functions using pointers
3. Flexibility in handling arrays and structures
4. Reduced time and space complexity